Chap-1 : Introdution & motivations

Année universitaire :2024/2025

Plan

- 1. Qu’est-ce que l’algorithmique ?
- 2. Pourquoi on calcule la complexité d’un algorithme?
- 3. Comment calculer la complexité d’un algorithme?
- 4. Etude d’exemples de problèmes

1. Qu’est-ce que l’algorithmique ?

1.1 Définition (Algorithme).

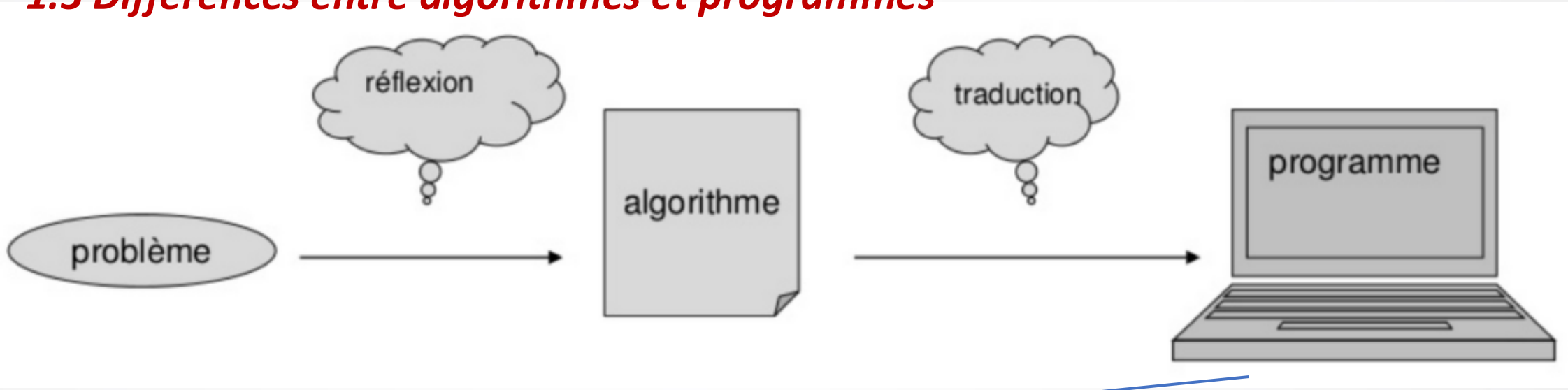
Un algorithme est une suite finie d’opérations élémentaires constituant un schéma de calcul ou de résolution d’un problème.

1.2 Définition (Algorithmique)

L’algorithmique désigne le processus de recherche d’algorithme.

1. Qu’est-ce que l’algorithmique ?

1.3 Différences entre algorithmes et programmes

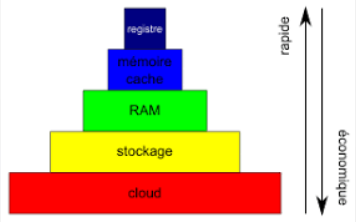


Condition d'exécution programme

Langage de programme

Allocation de mémoire

Puissance de calculateur…



1. Qu’est-ce que l’algorithmique ?

1.3 Différences entre algorithmes et programmes

- Un **programme** est la réalisation (l’implémentation) d’un algorithme au moyen d’un langage donné (sur une architecture donnée).

- Il s’agit de la mise en œuvre du principe.

- Par exemple, lors de la programmation on s’occupera parfois, explicitement de la gestion de la mémoire (allocation dynamique en C) qui est un problème d’implémentation ignoré au niveau algorithmique.

1. Qu'est-ce que l'algorithmique ?

1.4 Qualités exigées d'un bon algorithme

Un bon algorithme doit satisfaire les qualités suivantes:

- ***Correct:*** Il faut que le programme exécute correctement les tâches pour lesquelles il a été conçu
- ***Complet:*** Il faut que le programme considère tous les cas possibles et donne un résultat dans chaque cas.
- ***Efficace:*** Il faut que le programme exécute sa tâche avec efficacité c'est à dire avec un coût minimal. Le coût pour un ordinateur se mesure en termes de temps de calcul et d'espace mémoire nécessaire.

→**La complexité :** mesurer le coût : nombre d'opérations élémentaires

2. Pourquoi on calcule la complexité d'un algorithme?

- **Exemple introductif** : Chercher un élément dans un tableau trié

```
Algorithme RechercheSéquentielle(T, n, E)

    Pour i allant de 0 à n-1 faire
        Si T[i] = E alors
            Retourner i
        Fin Si
    Fin Pour

    Retourner -1  // Si l'élément n'est pas trouvé
Fin Algorithme
```

E= 77

45	10	4	77	-3

Complexité : O(n)

```
Algorithme RechercheDichotomique(T, n, E)

   Début ← 0
   Fin ← n - 1

   Tant que Début ≤ Fin faire
      Milieu ← (Début + Fin) // 2  // Calculer l'indice du milieu
      Si T[Milieu] = E alors
         Retourner Milieu  // L'élément est trouvé
      Sinon si T[Milieu] < E alors
         Début ← Milieu + 1  // Rechercher dans la moitié droite
      Sinon
         Fin ← Milieu - 1  // Rechercher dans la moitié gauche
      Fin Si
   Fin Tant que

   Retourner -1  // L'élément n'est pas trouvé
Fin Algorithme
```

E= 77

77	45	10	4	-3

Complexité : O(logn)

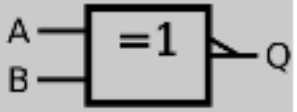
La recherche dichotomique est un algorithme très performant avec une complexité temporelle logarithmique, ce qui en fait une solution efficace pour les tableaux triés, bien plus rapide que la recherche séquentielle qui a une complexité de O(n)

2. Pourquoi on calcule la complexité d’un algorithme?

- Calculer la complexité d’un algorithme permet de comprendre son efficacité et ses limitations.
- Il est essentiel d'examiner la complexité en temps pour avoir une vision sur la performance de l'algorithme.

3. Comment calculer la complexité d'un algorithme?(1)

3.1. Identifier les opérations de base (Operations élémentaires)

Opérations élémentaires	Exemple
Une comparaison, un test, une opération logique	Var1>Var2 
Une opération arithmétique, une fonction mathématique	Var1+Var2
L'accès à un élément d'une structure de donnée (liste, chaine, etc..), sa création, sa modification.	strcture1.atribut1
L'affectation de variable	Var1=2 score ← score + 10
Saisie/impression/retour d'une donnée à l' écran, ou dans un fichier.	Ecrire(Var1) variable 1 0

3. Comment calculer la complexité d'un algorithme?(2)

3.2 Analyser le nombre d'opérations :

Nombre d'itérations dans le cas d'algorithme itératif

3.3 Notations asymptotiques « notation de Landau »

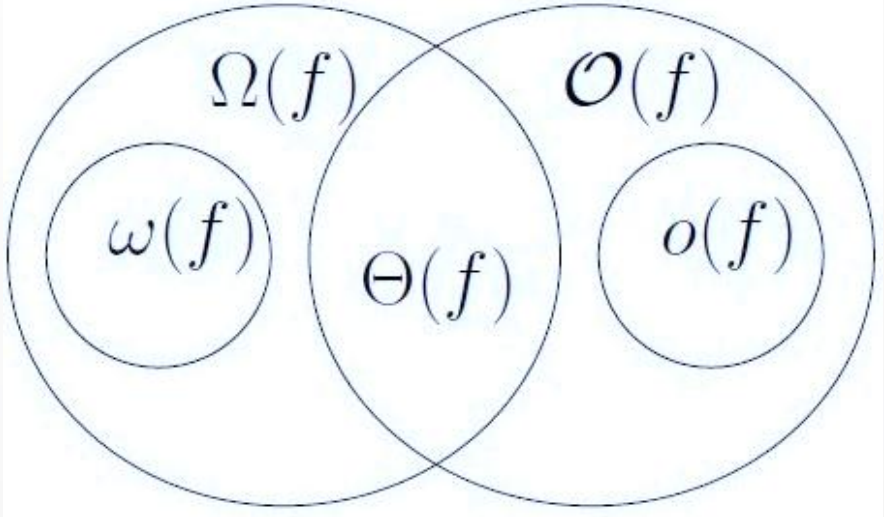
3.4 Déterminer une formulation de complexité :

cas d'algorithme récursif

3. Comment calculer la complexité d'un algorithme?(3)

3.3 Notations asymptotiques « notation de Landau » (1)

- Quand nous calculerons la complexité d'un algorithme, nous ne calculerons généralement pas sa complexité exacte, mais son ordre de grandeur.
- Pour ce faire, nous avons besoin de notations asymptotiques connues sous le nom de « notation de Landau » (Mathématicien Allemand).



3. Comment calculer la complexité d'un algorithme?(4)

3.3 Notations asymptotiques « notation de Landau » (2)

$$\mathrm{O}: \quad f = \mathrm{O}(g) \quad \Leftrightarrow \quad \exists n0, \exists c \geq 0, \quad / \quad \forall n \geq n0, \quad f(n) \leq c.g(n)$$

- On dit aussi que f « est dominée asymptotiquement par g »
- Ou que g « est un majorant presque partout de f »

Exemples

- $n = O(n)$, $2n = O(3n)$, $n+2 = O(n)$ (Il suffit de prendre $n0 = 2$ et $c = 2$)
- sqrt(n) = $O(n)$, $\log(n) = O(n)$, $n = O(n^2)$.

3. Comment calculer la complexité d'un algorithme?(5)

3.3 Notations asymptotiques « notation de Landau » (3)

$$\Omega : \qquad f = \Omega(g) \qquad \Leftrightarrow \qquad g = \mathrm{O}(f)$$

- Cela signifie que g « est majorée par f »

$$o : \qquad f = o(g) \qquad \Leftrightarrow \qquad \forall c \geq 0, \exists n0 \quad / \quad \forall n \geq n0, \qquad f(n) \leq c.g(n)$$

- Ceci signifie que : f « est négligeable devant g »

Exemples

- sqrt(n) = $o(n)$, log(n) = $o(n)$, $n = o(n^2)$, log(n) = o(sqrt(n)).

3. Comment calculer la complexité d'un algorithme?(6)

3.3 Notations asymptotiques « notation de Landau » (4)

- Ceci signifie que : $\Theta : \quad f = \Theta(g) \Leftrightarrow f = \mathrm{O}(g) \quad et \quad g = \mathrm{O}(f)$

$$f = \Theta(g) \Leftrightarrow \quad \exists c, d \in R^{+}, \exists n0 / \forall n > n0 \quad d.g(n) \leq f(n) \leq c.g(n)$$

- Ce qui veut dire que chaque fonction est un majorant de l'autre ou encore que les deux fonctions sont « de même ordre de grandeur asymptotique »

Exemples : n+log(n) = Θ (n+sqrt(n)).

Remarques

- On peut considérer que :
- $\mathrm{O}(g)$ est une borne supérieure
- $\Omega(g)$ est une borne inférieure
- $\Theta(g)$ est une borne exacte. Celle-ci est donc plus précise que les précédentes

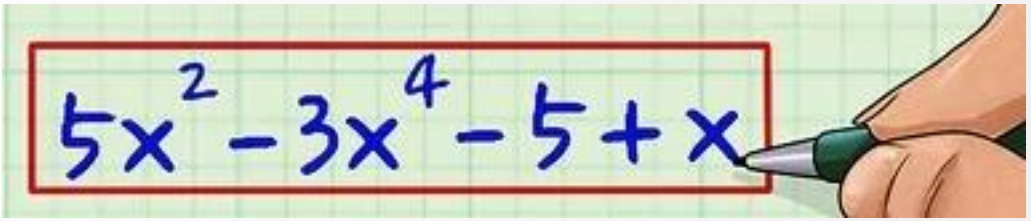
2. Problème_1 : Valeur d'un polynôme

On voudrait élaborer un algorithme permettant de calculer pour une valeur X donnée de type **REAL** la valeur numérique d'un polynôme de degré n:

$$P(X) = a_n X^n + a_{n-1} X^{n-1} + \ldots + a_1 X + a_0$$

Données : n et les coefficients $a_n, \ldots, a_0$ et x

Objectif : calcul de P(X)



2. Problème_1 : Valeur d'un polynôme

2.1 Algorithme_1 (trivial)

```
begin
   P=0
   for k=0 to n do
      P = P+ a_k*X^K
   endfor
end
```

2. Problème_1 : Valeur d'un polynôme

2.2 Algorithme_2 (sans puissance)

```
begin
   XP=1 ;     P =0
   for K=0 to N do
      P = P+ XP*a_k
      XP = XP * X
   endfor
end
```

2. Problème_1 : Valeur d'un polynôme

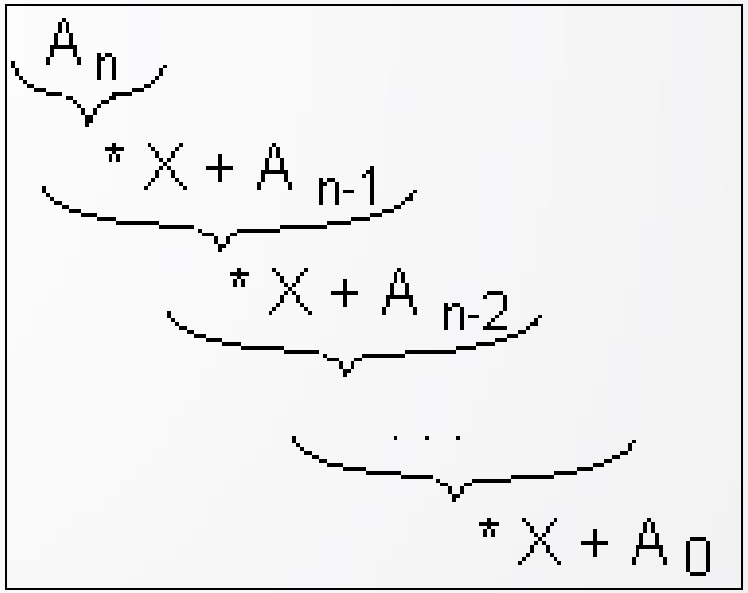
2.3 Algorithme_3 (Schéma de Horner)

- Le schéma de Horner est illustré par la figure ci-contre :

  $P(x) = (....(((a_n x+a_{n-1})x+a_{n-2})x+a_{n-3}).....)x+a_0$

  ➔ l'algorithme

```
begin
    P = a_n
    for k = n-1 to 0 step –1 do
        P = P*X + a_k
    endfor
end
```



…. Peut-on faire mieux ?

3. Problème-2: La plus grande somme contiguë

- Le but est d'élaborer un algorithme permettant de calculer la plus grande somme contiguë d'une séquence d'entiers placés dans un tableau.

- Étant donné un tableau $A[0..n\text{-}1]$ de n nombres entiers positifs ou négatifs, comment déterminer la valeur maximale des sommes correspondantes aux sections de la forme $A[i..j[$. On notera que 0 est la somme de n'importe quelle section vide $A[i..i[$.

- Par exemple, si le tableau contient les dix éléments

i	0	1	2	3	4	5	6	7	8	9
A	31	-41	59	26	-53	58	97	-93	-23	84

- la somme maximale est $A[2..7[= A[2..6] = 187$.

3. Problème-2: La plus grande somme contiguë

3.1 Algorithme_1 (naïf)

L'idée la plus simple est de calculer toutes les sommes et de rechercher la plus grande, par un algorithme du style.

```
Algorithme_1 (int n , int a[ ])
        Maximum=0;
        For i=0 to n-1 do
                For j=i to n-1 do
                        Somme=0
                        For k=i to j do
                                Somme= Somme + a[k]
                        Endfor
                        If (Somme > Maximum) then
                                Maximum = Somme
                Endfor
        Endfor
        Return Maximum
End_algorithme
```

Il est facile de voir que la comparaison entre Maximum et Somme est effectuée $O(n^3)$ fois.

… Peut-on mieux faire ?

3. Problème-2: La plus grande somme contiguë

3.2 Algorithme_2

Une meilleure idée consiste à ne pas recalculer systématiquement les sommes mais à utiliser la relation :

$$\sum_{k=i}^{j+1} a_k = \sum_{k=i}^{j} a_k + a_{j+1}$$

D'où le nouveau programme

```
Algorithme_2 (int n , int a[ ])
        Maximum=0;
        For i=0 to n-1 do
                Somme=0
                For j=i to n-1 do
                        Somme= Somme + a[j]
                        If (somme > maximum) then
                                Maximum = Somme
                Endfor
        Endfor
        Return Maximum
End_algorithme
```

Cette version n'effectue plus que $O(n^2)$ additions.

…. Peut-on faire mieux ?

3. Problème-2: La plus grande somme contiguë

3.3 Algorithme_3 :

On peut essayer une approche **dichotomique** en coupant le tableau en **deux parties de tailles équivalentes** et en disant que la somme maximale est réalisée soit totalement dans la partie droite, soit totalement dans la partie gauche, soit à cheval.

La valeur recherchée est nécessairement la plus grande des trois sommes. Cette idée nous conduit à l'algorithme récursif suivant :

3. Problème-2: La plus grande somme contiguë

On invoque cette fonction par l'appel **algorithme_3 (a, 0, n-1).**

```
Algorithme_3(int a[ ], int g, int d)
    int m, max1, max2, maximum ;
    int somme, i;
    if (g > d ) then
        return 0;
    endif
    if (g ==d ) then
        if (a[g] > 0 )
                return a[g];
                else
                return 0;
    endif
    m= (g+d)/2;
    max1=0;somme = 0;
    for i=m to g step -1 do
        somme = somme +a[i];
        if (somme >  max1)
            max1= somme;
    endfor
    max2= 0 ;                    somme =0;
    for i=m+1 to d do
        somme= somme+ a[i];
        if (somme > max2) then
            max2= somme;
    endfor
    maximum = max1 + max2;
    somme= algorithme_3 (a, g, m);
    if ( somme > maximum ) then
        maximum =somme;
    endif
    somme= algorithme_3 ( a, m+1 , d);
    if ( somme > maximum ) then
        maximum =somme;
    endif
    return maximum;
End_algorithme
```

3. Problème-2: La plus grande somme contiguë

Complexité de l'algorithme

- Cet algorithme est de type « diviser pour régner ». Le nombre de comparaisons effectuées à l'intérieur de la fonction récursive est égal à (d-g) fois. Il est donc **O(n).**

- L'équation récurrente de sa complexité est donc:

- On démontrera plus loin en utilisant le théorème qui sera présenté au chapitre suivant que :

  $T(n) = O(n \log(n))$

  Car :

  $T(n) = 2T(n/2) + O(n)$

… Peut-on mieux faire ?

3. Problème-2: La plus grande somme contiguë

3.4 Algorithme_4

- Mais on peut encore mieux faire.

- L'idée cette fois est de parcourir une seule fois le tableau et de maintenir simultanément deux maximums : la plus grande somme « max1 » et un « max2 » qui est remis à zéro chaque fois qu'il devient négatif.

- La plus grande somme recherchée est égale soit à « max1 » soit à « max2 »

3. Problème-2: La plus grande somme contiguë

d'où l'algorithme

- Il est clair que cet algorithme est en $O(n)$.

... Peut-on mieux faire ?

- La réponse est non car tout algorithme résolvant ce problème est obligé de parcourir l'intégralité des données, donc a une complexité au moins linéaire.

```
int algo_4(int a[ ], int n)
        int max1 = 0, max2 = 0, i ;
        for i = 0 to  n-1 do
                max2 = max2 + a[i] ;
                if (max2 < 0) then
                        max2 = 0 ;
                endif
                if (max2 > max1) then
                        max1 = max2 ;
                endif
        endfor
        return max1  ;
End_algorithme
```

3. Problème-2: La plus grande somme contiguë

- Sur un PC à 400MHz, pour un tableau de taille 10^7 Jon Bentley, (Programming Pearls, Addison-Wesley, 2000)) donne l'estimation suivante des temps d'exécution:

Algorithme	1	2	3	4
Temps	41000 ans	1,7 semaines	11 secondes	0,48 seconde

4. Problème_3 : calcul de x^n

Données : Un entier naturel *n* et un réel *x*.

Objectif : Calculer x^n.

Idée : Nous partons de $y1 = x$. Nous allons construire une suite de valeurs $y1, ..., ym$ telle que la valeur yk soit obtenue par multiplication de deux puissances de *x* précédemment calculées : $yk=yu * yv$, avec $1<=u$, $v < k$, k *appartient à* $[2,m]$.

But : $ym = x^n$.

Le **coût** de l'algorithme sera alors de m-1 = nombre de multiplications faites pour obtenir le résultat recherché.

4. Problème_3 : calcul de x^n

4.1 Algorithme trivial

$y_i = y_{i-1} * y_1$, i appartient à $[2,n]$. Résultat : $y_n = x^n$.

Coût = n-1 multiplications.

```
Algorithme
y = x
For i =2 to  n  do
   y = y * x
End_for
Return y
End_Algorithme
```

4. Problème_3 : calcul de x^n

4.2 Méthode binaire ou méthode de « la chaîne Chinoise »

- **Historique :**
- Cette méthode a été présentée 200 avant J.C. en Inde (ou en Chine), mais il semblerait qu'il ait fallu attendre un millénaire avant que cette méthode ne soit connue en dehors de l'Inde [Knuth].

Algorithme	**Illustration avec x^{23}**
1. Écrire n sous forme binaire	1. $n = 23 = 10111$
2. Remplacer chaque : – « 1 » par la paire de lettres « SX » ; – « 0 » par la lettre « S ».	1 0 1 1 1 2. SX S SX SX SX
3. Éliminer la paire « SX » la plus à gauche.	3. S SX SX SX
4. Résultat : Une chaîne de caractères contenant les lettres S et X.	4. Résultat : SSXSXSX
5. Calcul de x^n : - partir de x et parcourir la chaîne de caractères - à chaque S ➔ « élever au carré » ; - à chaque X ➔ « multiplier par x ».	5. Nous partons de x et nous obtenons successivement : $x, x^2, x^4, x^5, x^{10}, x^{11}, x^{22}, x^{23}$. Nous sommes donc capables de calculer x^{23} en 7 multiplications au lieu de 22

4. Problème_3 : calcul de x^n

Explication de la méthode

– Écriture binaire de *n* :

$$n = \sum_{i=0}^{p} a_i 2^i$$

– Plaçons nous au cours du calcul de puissances de *x*.

Soit *j* le dernier bit de la représentation binaire de *n* qui a été «décodé»

et

soit *yj* le dernier résultat obtenu. (Initialement, *j* = *p* et *yp* = *x* = *xap*).

j varie de p à 0.

4. Problème_3 : calcul de x^n

– **Deux cas sont possibles pour *aj*-1 :**

- *1er cas : aj*-1 = 1 alors *aj*-1 est remplacé par SX, nous élevons *yj* au carré puis multiplions le résultat par *x*. Le nouveau résultat est : $y_{j-1} = y_j^2 x$

- 2ème cas : *aj*-1 =0alors *aj*-1 est remplacé par S et nous élevons simplement *yj* au carré. Le nouveau résultat est : $y_{j-1} = y_j^2$

Dans tous les cas nous avons : $y_{j-1} = y_j^2 x^{a_{j-1}}$

D'où : $y_{p-1} = y_p^2 x^{a_{p-1}} = (x^{a_p})^2 (x^{a_{p-1}}) = (x^{2a_p})(x^{a_{p-1}}) = (x^{2a_p + a_{p-1}})$

4. Problème_3 : calcul de x^n

- Par récurrence, on peut montrer que :

$$y_0 = x^{\sum_{i=0}^{p} a_i 2^i} = x^n$$

4. Problème_3 : calcul de x^n

Complexité (coût)

- **Notes**

  les nombres dont la représentation binaire ont exactement *p* chiffres forment exactement l'intervalle : $[2^{p-1}, 2^p - 1]$

  Nombres de chiffres dans l'écriture binaire de $n = 1+[\log_2 n]$. Notons v(n) le nombre de «1» dans l'écriture binaire de n.

➔ Nombre d'opérations effectuées =

$(1+\log_2 n)-1$ élévations au carré (ne pas oublier l'étape 3) ;
v(n)-1 multiplications par x (ne pas oublier l'étape 3).

4. Problème_3 : calcul de x^n

- Soit en tout : $T(n) = \log_2 n + v(n) - 1$ multiplications.
- Sachant que : $1 <= v(n) <= \log_2 n$ ➔ $\log_2 n <= T(n) <= 2\log_2 n$

- Pour n = 1000, l'algorithme trivial effectue 999 multiplications, et la méthode binaire moins de 20.

….. Peut-on mieux faire ?

4. Problème_3 : calcul de x^n

- Prenons le cas n = 15.

1- n = 1111

1	1	1	1
SX	SX	SX	SX
	SX	SX	SX

4- Nous partons de x et nous obtenons successivement : x^2, x^3, x^6, x^7, x^{14}, x^{15}.

- Nous sommes donc capables de calculer x^{15} en 6 multiplications.

- Autre schéma de calcul : x^2, x^3, x^6, x^{12} et faisons $x^{15} = x^{12} \times x^3$

➔ Nous obtenons ainsi x^{15} en 5 multiplications

➔ la méthode binaire n'est donc pas *optimale* (c'est-à-dire que l'on peut faire mieux).

4. Problème_3 : calcul de x^n

4.3 Algorithme des facteurs

Algorithme

$$x^n = \begin{cases} x & si \quad n = 1 \\ x^{n-1}.x & si \quad n \quad premier \\ (x^p)^{n'} & si \quad n = p.n' \quad avec \quad p \quad le \quad ppdp \quad de \quad n \end{cases}$$

ppdp : le plus petit diviseur premier

4. Problème_3 : calcul de x^n

- **Illustration avec *n* = 15**

1. 15 = 3x5, 3 étant le plus petit diviseur (facteur) premier de 15. Donc $x^{15} = (x^3)^5$.

Nous réappliquons l'algorithme pour calculer x^3 et y^5, où $y = x^3$.

2. Calcul de x^3 :

(a) 3 est premier. Donc $x^3 = x^2 .x$

Nous réappliquons l'algorithme pour calculer x^2.

(b) 2 est premier. Donc $x^2 = x.x$

(c) Finalement, x^3 est calculé comme suit : $x^3 = x.x.x$, soit en deux multiplications.

3. Calcul de y^5 :

(a) 5 est premier. Donc $y^5 = y^4.y$ Nous réappliquons l'algorithme pour calculer y^4.

(b) 4 = 2x2, où 2 est le plus petit facteur premier de 4. Donc $y^4 = (y^2)^2$.

(c) Finalement y^5 est calculé comme suit : $t = y.y$, $u = t.t$, $y^5 = u.y$, soit en 3 multiplications.

4. Finalement, x^{15} est calculé en 5 multiplications.

… Peut-on faire mieux ? Oui... ➔ L'algorithme de l'arbre (à découvrir)

4. Problème_3 : calcul de x^n

➔ Nous avons affaire à un problème simple, que tout le monde sait résoudre, mais qu'il est très difficile de résoudre efficacement...

5. Problème_4 : Calcul de la Trans. de Fourier »

- Transformée de Fourier 1-D continue (rappel) :
- Si *f*(x) est continue et intégrable, alors la transformée de Fourier continue est définie par :

$$F(u)=\int_{-\infty}^{+\infty} f(x)e^{-2j\pi ux}dx$$

et l'inverse

$$f(x)=\int_{-\infty}^{+\infty} F(u)e^{2j\pi ux}du$$

La TF discrète est donnée par :

$$F(u)=1/N\sum_{x=0}^{N-1} f(x)e^{-2j\pi ux/N}$$

et

$$f(x)=\sum_{u=0}^{N-1} F(u)e^{2j\pi ux/N}$$

- **Données :** N échantillons de la fonction *f(i), pour i=0,1,…, n*

- **Objectifs :** Calculer *F(u) pour une valeur de u donnée*

5. Problème_4 : Calcul de la Trans. de Fourier »

5.1 Algorithme trivial

```
pin= 6.28*u/n;
Fu=0
For k = 0 to n do
   Fu = Fu + f(k)*exp(-j*pin*k)                    // opérations complexes
Endfor
```

Coût de l'algorithme

(n+1) additions complexes
(n+1) multiplications complexes
(n+1) multiplications réelle
(n+1) calcul d'une exponentielle

5. Problème_4 : Calcul de la Trans. de Fourier »

5.2 Algorithme de Tookey Kooley

On pose $W_N^{ux} = \exp(-2i\pi ux/N)$ et $W_N = \exp(-2i\pi/N)$

→

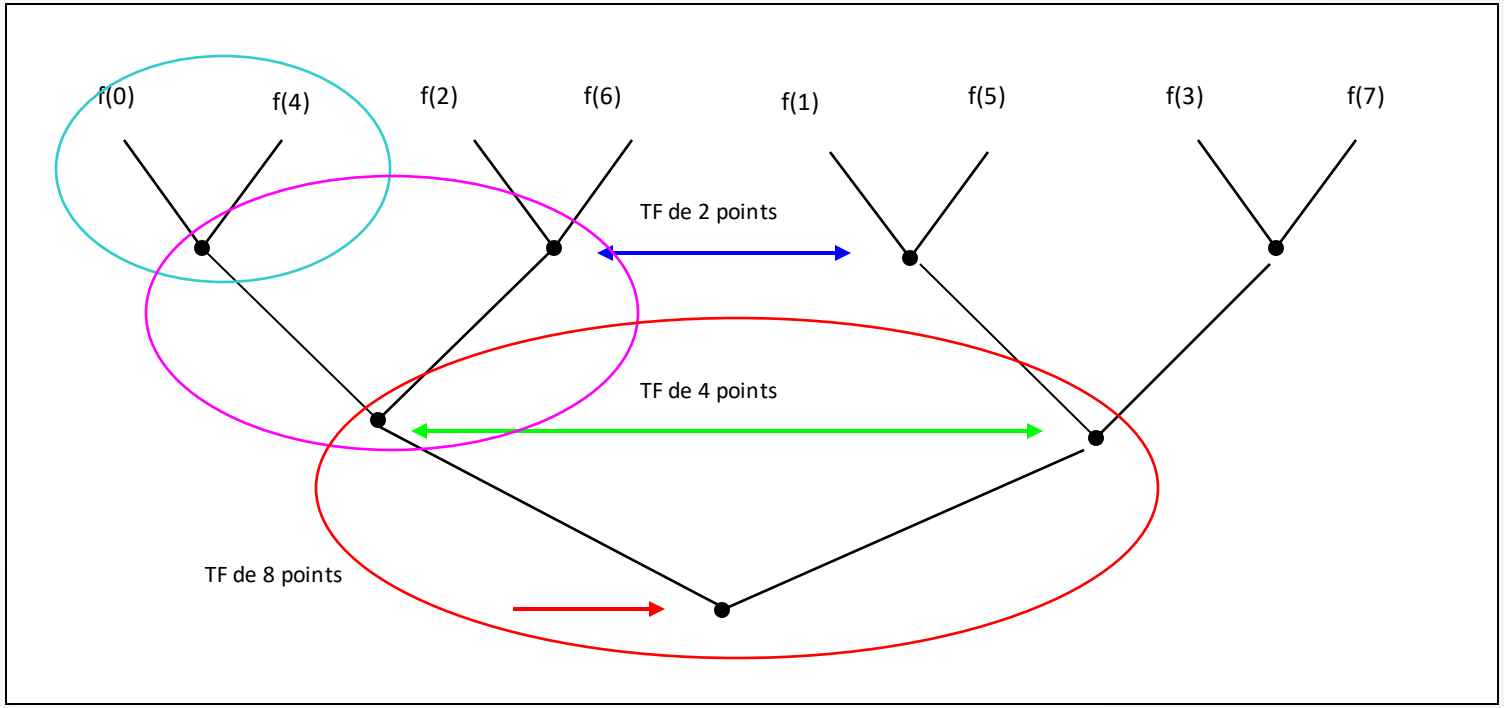
$$F(u) = 1/2\left\{1/M\sum_{x=0}^{M-1} f(2x)W_{2M}^{u(2x)} + 1/M\sum_{x=0}^{M-1} f(2x+1)W_{2M}^{u(2x+1)}\right\}$$

$$F(u) = 1/2\left\{F_{paire}(u) + F_{impaire}(u)W_{2M}^{u}\right\}$$

➔ Par récursivité, le calcul de la TF se ramène au calcul de la TF de 2 échantillons

5. Problème_4 : Calcul de la Trans. de Fourier »

- ***Exemple*** *: Fonction à 8 échantillons*



- ➔ *il suffit d'avoir une fonction récursive qui calcule la TF de deux points*

5. Problème_4 : Calcul de la Trans. de Fourier »

Coût de l'algorithme

On démontre que pour calculer la TF d'une fonction à n échantillons :

T(n) est fonction de $n\log_2(n)$ (voir théorème plus loin)

6. Conclusion

- Existence de plusieurs solutions pour un problème donné
- Ces solutions peuvent être correctes mais n’ont pas la même efficacité (coût)
- Avant de proposer un algorithme, il faut estimer son coût
- Et se poser la question de l’existence d’une autre solution moins coûteuse